

Experiment 7 — Reaching the vendor kernels

Mathilda — execution-speed experiments

Contents

Experiment 7 — Reaching the vendor kernels	1
Hypothesis	1
What was built	1
Results	1
The scratch pool: built, measured, rejected	2
Still open	3
Why Mathilda is not the fastest here, and what it would take	3

Experiment 7 — Reaching the vendor kernels

Date: 2026-07-30 · Commits: 57b3370 (route Dot/QR), 83ba959 (the boundary), 87a70db (the rejected scratch pool) · Code: `src/linalg/` · Result: Det overtakes Mathematica; and one hypothesis disproved

Common method in [README.md](#).

Hypothesis

Dense linear algebra is the one area where every system should be equal, because every system calls the same library. Mathilda, Mathematica and NumPy all link Apple Accelerate on this host. If $A \cdot A$ is slower in Mathilda than in Mathematica, the difference is not the arithmetic — it is what happens on either side of the call.

The first HPC sweep said the gap was 1.24× to 12.6×, widening exactly where Mathilda stopped calling LAPACK.

What was built

1. Route the remaining heads to BLAS/LAPACK (57b3370). Dot now covers every shape BLAS has — dgemm, dgemv in both orientations, ddot — rather than falling to a hand-written loop for the vector cases. QRDecomposition routes to dgeqrf + dorgqr.
2. Stop converting element by element (83ba959). `na_load_matrix` converted Mathilda's row-major buffer to LAPACK's column-major layout one `ndt_get` at a time, and `na_build_matrix` converted back the same way. Both are now a cache-blocked transpose, or a memcpy where no transpose is needed.

Results

Benchmark	Mathilda	Mathematica 14.0	NumPy / Python	vs WL	vs NumPy
Matrix multiply, 1000x1000	7.42 ms	6.46 ms	7.07 ms	1/1.15x	1/1.05x
Det, 500x500	1.41 ms	1.56 ms	1.42 ms	1.11x	1.01x
LinearSolve, 1000x1000	14.45 ms	6.16 ms	14.44 ms	1/2.35x	1/1.00x

Benchmark	Mathilda	Mathematica 14.0	NumPy / Python	vs WL	vs NumPy
Inverse, 500x500	7.47 ms	3.23 ms	5.32 ms	1/2.32x	1/1.40x
SingularValueDecomposition, 300x300	52.60 ms	8.47 ms	19.86 ms	1/6.21x	1/2.65x
Eigenvalues, 300x300 symmetric	22.38 ms	2.94 ms	3.91 ms	1/7.61x	1/5.72x
QRDecomposition, 500x500	59.58 ms	4.39 ms	18.75 ms	1/13.59x	1/3.18x

All three link Apple Accelerate, so on the first four rows the arithmetic is byte-identical and the spread is entirely marshalling. NumPy sits between the two CAS on Inverse and beside Mathilda on LinearSolve and matrix multiply.

Det is now faster than Mathematica, and matrix multiply is close: both reach the same Accelerate kernels and the conversion into them is no longer element-wise.

The spread widens exactly where Mathilda stops using LAPACK. QRDecomposition, Eigenvalues and SingularValueDecomposition run in-house numeric kernels. Eigenvalues uses Mathilda's own QR iteration, kept in preference to LAPACK because the eigenvalue ordering convention ($|\lambda|$ ties broken by position) cannot be reproduced from LAPACK output without risking parity — a deliberate trade, recorded in [NDARRAY_REDUCTIONS_COMPARISON.md](#).

QRDecomposition went from 18.7x to 14.0x: LAPACK's own factorisation is only a few milliseconds of it. The rest is the boundary, plus ~24 ms materialising the $\{q, r\}$ pair into plain Lists, which the no-nesting invariant requires and Mathematica does not do.

The scratch pool: built, measured, rejected

This is the most useful result in the experiment, and it is a negative one.

The hypothesis. A flat profile of LinearSolve attributed ~27% of samples to `mach_vm_map/mach_vm_deallocate/madvise`. The obvious reading: the 8 MB column-major load buffer is being allocated and freed on every call, and a size-keyed scratch pool would remove it.

It was built properly. Header-prefixed sizes, mutex-guarded, opt-in through `na_load_matrix_scratch` so the ~116 plain `free()` calls across the linalg bridges could not silently become heap bugs. The three measured paths were converted.

It changed nothing. Min-of-five, two runs each:

	with pool	without
LinearSolve 1000 ²	15.45, 15.42 ms	15.56, 15.12 ms
Inverse 500 ²	8.24, 8.94 ms	8.49, 8.02 ms
Det 500 ²	1.75, 1.54 ms	1.44, 1.63 ms

Why the hypothesis was wrong. A flat "top of stack" profile attributes a sample to a symbol, not to a culprit. Reading the call graph shows those allocations belong to Accelerate:

```
30 ??? (in libBLAS.dylib)
: 29 _szone_free (in libsystem_malloc.dylib)
: | 13 free_large
: | + 13 madvise
```

That is libBLAS's own per-call internal workspace, inside its threaded implementation. Nothing Mathilda allocates, and nothing a pool on our side can reach. Mathilda's own 8 MB load buffer is one malloc/free pair per call and was never the problem.

It was reverted, not kept: ~150 lines carrying a class of heap bug for no measured benefit. The plan records the experiment so it is not repeated.

A flat profile says where the time goes; only the call graph says whose it is.

That rule is now in `tasks/lessons.md`. It is the second time in these experiments that a measurement pointed at an innocent symbol — the other is the N-body step in [HPC_SWEEP_2_APPLICATIONS.md](#), where profiling the slow call implicated Outer and the arithmetic, and the actual cause was a return statement in the previous call.

Still open

- Eigenvalues and SingularValueDecomposition do not use LAPACK, by choice. Routing them requires reproducing the ordering convention, which is a parity problem rather than a performance one.
- The `{q, r}` materialisation at the QRDecomposition boundary is ~24 ms of the 59 ms and is a direct consequence of the no-nesting invariant (a List of buffers is the malformed shape the packing gate exists to prevent).

Why Mathilda is not the fastest here, and what it would take

This is the experiment with the largest remaining gaps in the suite, and the cleanest diagnosis: all three systems link the same Apple Accelerate BLAS, so on any row where Mathilda is far from the other two, the vendor kernel is simply not being reached.

row	Mathilda	best other	gap	reaches LAPACK?
Matrix multiply, 1000^2	7.42 ms	6.46 ms (WL)	1.15×	yes — dgemm
Det, 500^2	1.41 ms	1.42 ms (NumPy)	1.11× ahead	yes
LinearSolve, 1000^2	14.45 ms	6.16 ms (WL)	2.35×	yes, but one extra copy
Inverse, 500^2	7.47 ms	3.23 ms (WL)	2.32×	yes, but one extra copy
SingularValueDecomposition, 300^2	52.6 ms	8.47 ms (WL)	6.21×	no
Eigenvalues, 300^2 symmetric	22.4 ms	2.94 ms (WL)	7.61×	no
QRDecomposition, 500^2	59.6 ms	4.39 ms (WL)	13.59×	partial

The bottom three run in-house factorisations. That is not an oversight in every case — Eigenvalues' ordering convention cannot be reproduced from LAPACK's output without care, and getting it wrong is a parity break rather than a slowdown — but it is the whole of the gap.

The road to fastest

1. **QRDecomposition** → **dgeqrf + dorgqr** (13.6×, the largest single gap in the suite). Mathilda's convention returns `{q, r}` with `q` transposed relative to LAPACK's; that is one transpose at the boundary, and the in-house path stays as the fallback for the shapes LAPACK declines.
2. **Eigenvalues/Eigensystem** → **dsyevd/dgeev** (7.6×). The work here is the ordering and sign shim, not the call: LAPACK returns ascending eigenvalues where Wolfram returns them by decreasing magnitude, and eigenvector signs are arbitrary. Differential-test the shim against the existing implementation across the whole `eigen_tests` corpus before switching the default.
3. **SingularValueDecomposition** → **dgesdd** (6.2×). The same shape of job, with the same sign convention question for the singular vectors.
4. Remove the copy at the **LinearSolve/Inverse** boundary (2.3×). Both already reach LAPACK; both convert the matrix element by element on the way in. A packed float64 argument is already exactly the buffer LAPACK wants, so the copy is pure loss.

Items 1–3 are each "call the vendor routine and shim the convention", and together they would take this experiment from three rows at 6–14× behind to the whole group at parity. Item 4 is smaller and easier and should be done first as a warm-up, since it touches the same boundary code.

A rejected item, recorded so it is not proposed again: a per-thread scratch pool for LAPACK workspaces was built and measured, and made no difference at any size (see above). The allocation is not the cost.